# EAX x86 Register

## Meaning and History

Vladimir Keleshev • 2020-03-20

Usually, x86 tutorials don't spend much time explaining the historical perspective of design and naming decisions. When learning x86 assembly, you're usually told something along the lines: *Here's* `EAX`. *It's a register*. *Use it*.

So, what exactly do those letters stand for? E–A–X.

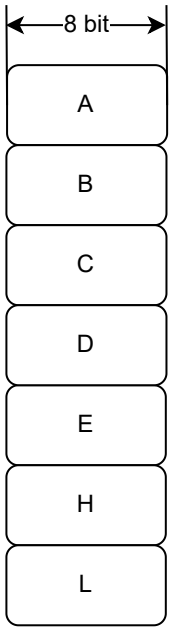
I'm afraid there's no short answer! We'll have to go back to 1972…

## *8008*

In 1972, after an odd sequence of events, Intel introduced the world's first 8-bit microprocessor, the *8008*. Back then, Intel was primarily a vendor of memory

chips. The 8008 was commissioned by the Computer Terminal Corporation (CTC) for their new Datapoint 2200 programmable terminal. But the chip was delayed and did not meet CTC expectations. So Intel added a few general-purpose instructions to it and marketed the chip to other customers.

8008 had *seven* 8-bit registers:




> "`A`" stood for *accumulator*, which was an implicit operand and return value of the arithmetic and logical operations.

You might think—*gee, seven is a very odd number of registers*—and would be right! The registers were encoded as three bits of the instruction, so it allowed for

eight combinations. The last one was for a pseudo-register called `M`. It stood for *memory*. `M` referred to the memory location pointed by the combination of registers `H` and `L`. `H` stood for *high-order* byte, while `L` stood for *low-order* byte of the memory address. That was the only available way to reference memory in 8008.

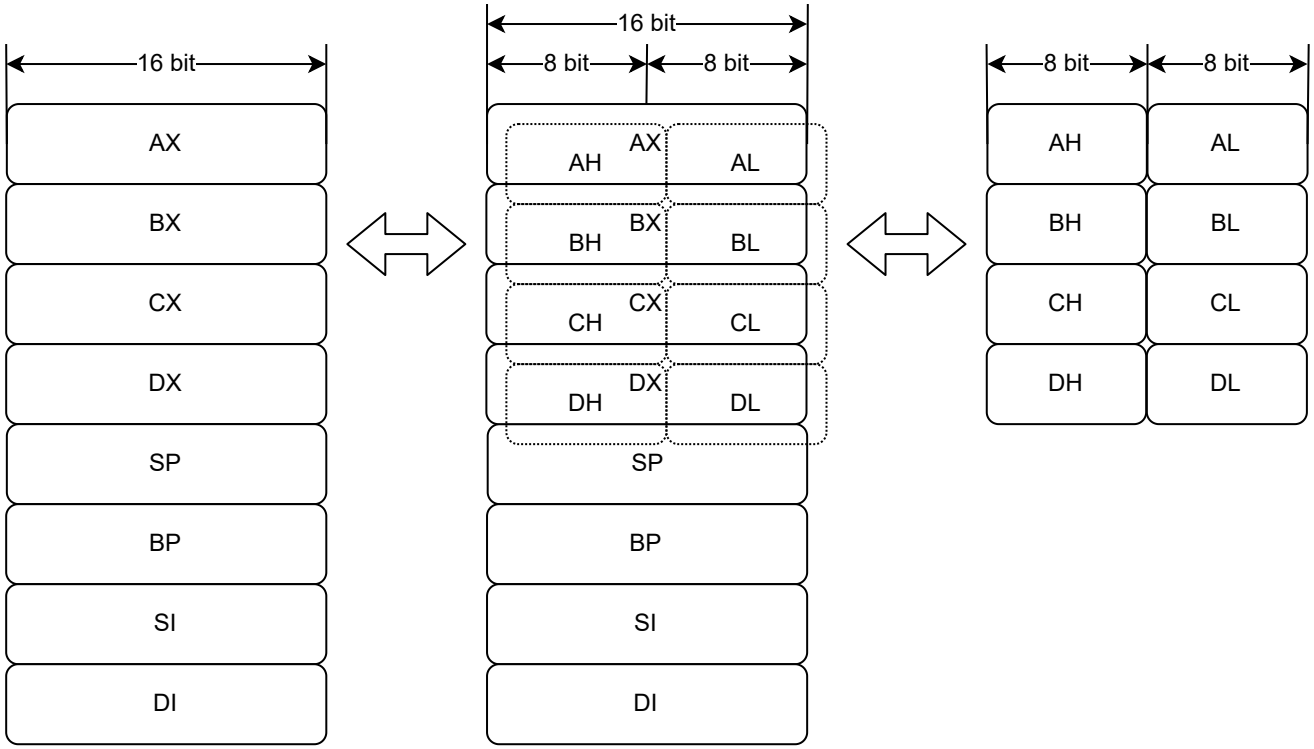
So, `A` was an accumulator, `H` and `L` were also used for addressing memory. However, `B`, `C`, `D`, `E` were completely generic and interchangeable.

## *8086*

In 1979, Intel was already a microprocessor company, and their flagship processor iAPX 432 is delayed. So as a stop-gap measure, they introduce 8086, a 16-bit microprocessor derived from 8080, which was itself derived from 8008.

To leverage its existing customer base, Intel made 8086 software-compatible down to 8008. A simple translator program would translate from 8008 assembly to 8086 assembly. For that to work well, 8086 instruction set architecture had to map well to 8008, inheriting many design decisions.

8086 had eight 16-bit registers and eight 8-bit registers, and they overlapped as follows:




8086 instructions had a bit flag that specified whether the three-bit encoding of a register referred to one of eight 8-bit registers, or to one of eight 16-bit registers.

As you can see from the figure above, data in the first four 16-bit registers could also be accessed by one of the eight 8-bit registers.

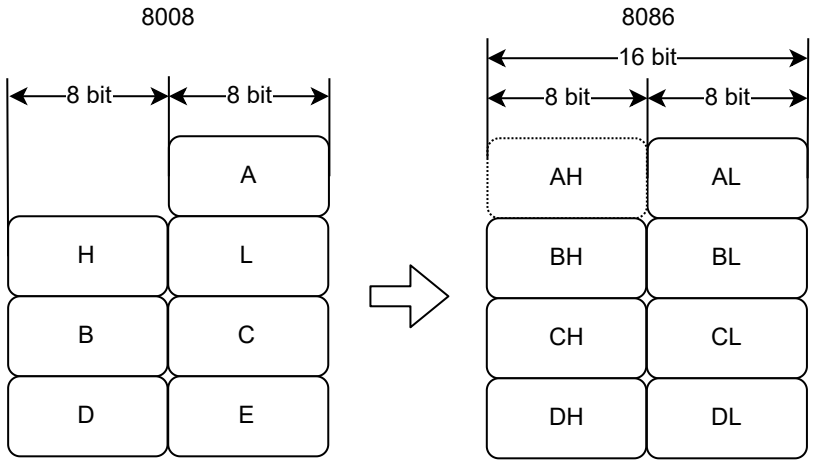
> `AX` was a 16-bit accumulator, while `AH` and `AL` could be thought of as 8-bit registers on their own or as a way to access the *high-order* and the *low-order* bytes of `AX`.
>
> The `X` in `AX` meant to be a placeholder that stood for both `H` and `L`.

> This is in a way similar to how much later the “x” in x86 was meant to refer to 8086, 80186, 80286, etc.

Since 8008 had seven 8-bit registers, they could be mapped well to the eight 8086 registers, with one to spare.

The `M` pseudo-register was not needed anymore since 8086 allowed for many memory addressing modes. Hence, it freed an encoding for an additional register.

In the following figure you can see how exactly the 8008 registers were mapped to the 8086 ones:




Even though many arithmetic and logical operations could work on any of these registers, none of the registers were truly generic at this point. Each had some instructions introduced that worked for one of the registers but didn’t work for

others. The mnemonics are: `BX` is *base register*, `CX` is *count register*, `DX` is *data register*, and `AX` is still the *accumulator*.

The new `SP` is *stack pointer*, `BP` is *base pointer*, `SI` is *source index*, `DI` is *destination index*. But we won't go into details about them here.

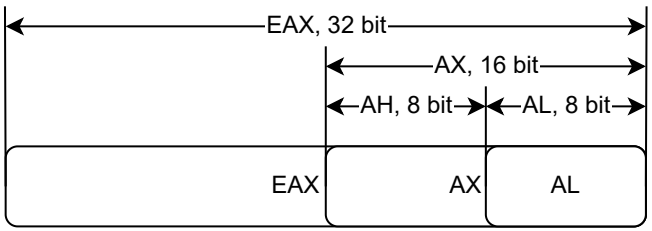
8086 also introduced the segment registers, but they were very much a separate beast. Segmented architecture deserves a story on its own, as it is the result of maintaining backward-compatibility with 8080.

## *x86*

In 1985 Intel introduced 80386, the first 32-bit processor in the x86 line. An early batch of processors had a defect in one of the 32-bit operations. They were marked "16-BIT S/W ONLY" and sold anyway.

Many new features were introduced, but 80386 continued to be *(mostly)* binary-compatible down to 8086.

The main registers were *extended* to 32 bits by adding an `E` prefix:



> `EAX` stood for *extended* `AX`. And `AX` now refers to the lower half of `EAX`, while `AH` and `AL` continue to refer to the two `AX` bytes.

And that's how `EAX` got its name.

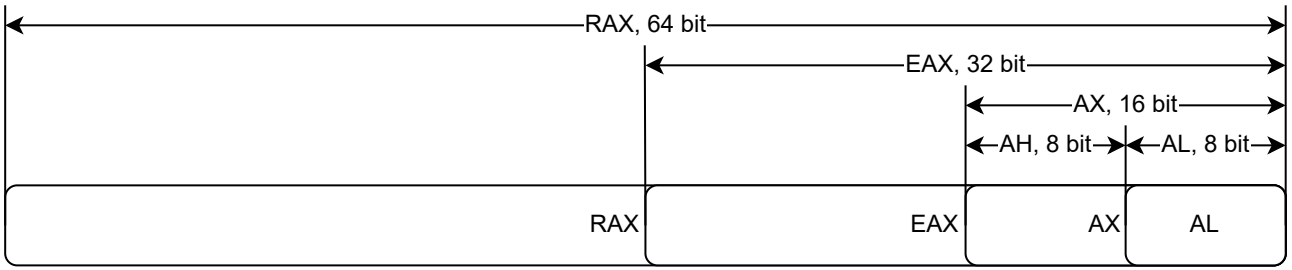
*But wait, there's more to the story!*

## *x86-64*

In 2003 AMD effectively takes over the architectural leadership and introduces the first 64-bit processor in the x86 lineage. In *legacy mode*, it is backward-compatible down to 8086.

The eight main registers are extended to 64 bits.

The extended registers get an `R` prefix that replaces the `E` prefix. So the accumulator is now referred to as `RAX`:

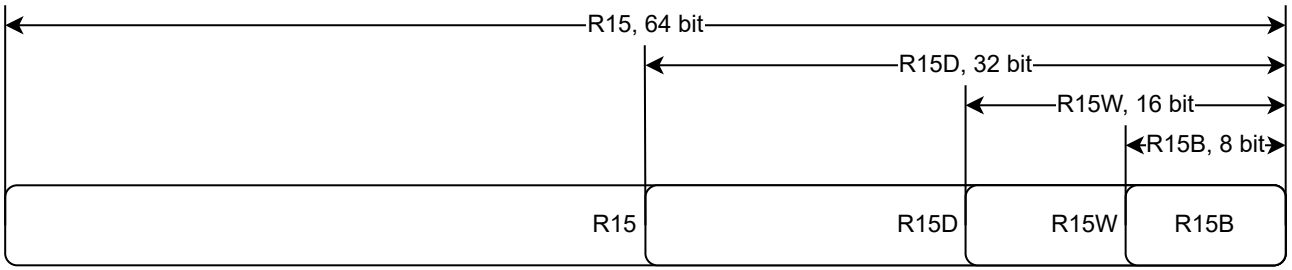


*Why `R`?*

Well, AMD wanted to streamline the register handling. They introduced eight new registers called `R8` to `R15`. They even discussed calling the extensions of the existing eight registers as `R0` to `R7`. But they recognized that many instructions have mnemonics that refer to one of the register letters like `A` or `B`. So they kept the original names, replacing `E` with `R`. That also provided at least some consistency with the new `R8`–`R15`.

> So `R` in `RAX` stood for *register*, and was a way to unify the naming to be more consistent with the new `R8`–`R15` registers.

The new registers also got their "narrow" versions. Take `R15`, for example:




⁂

And that, folks, was a quick history of the x86 accumulator! From an 8-bit `A` of 8008, to 16-bit `AX` of 8086, to 32-bit `EAX` of 80386, to 64-bit `RAX`.

## *Correction*

An earlier version of this blog post stated that the `X` in `AX` stood for e*X*tended.

Some of you pointed out that this was not quite right and that the `X` stood, in a way, for "pair". I must admit that, unlike for the rest of the article, I couldn't find a reference that authoritatively described the meaning of `X`. So, I decided to reach out to Dr. Stephen Morse, the architect of 8086.

With his permission, I include the response:

> *Vladimir,*
>
> *Your question is certainly pushing my memory about decisions that were made over 40 years ago. So the following is the best of my recollection and not necessarily 100% accurate.*
>
> *Prior to the 8086 the registers were single letters, e.g., A, B, C, D. Each was an 8-bit register. The 8086 had 16-bit registers that could be referenced either 8-bits at a time or all 16-bits at once. For example, we could reference the 8 high-order bits of the A register, the 8 low-order bits of the A register, or the entire 16 bits of the A register. The nomenclature of the*

> *first two were chosen to be AL and AH, where the L/H designated the low-order or the high order half. Now we needed a term to designate the full 16 bits. So the letter X was selected. The X was simply an arbitrary letter that combined both L and H – sort of like the use of X in algebra to designate the unknown. There really wasn't that much thought given as to what X stood for (if anything) – it was just a letter that was needed to identify the General Registers (AX, BX, CX, DX), as opposed to the Pointer and Index Registers (SP, BP, SI, DI), and the Segment Registers (CS, DS, ES, SS).*
>
> *– Steve Morse*

## *References*

Intel Microprocessors: 8008 to 8086 by Stephen Morse.